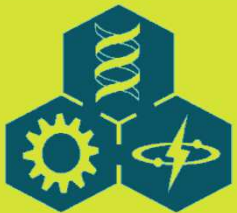
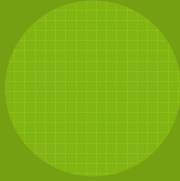


LECTURE 7:

PARAMETER ESTIMATION





OUTLINE

7.1 The Problem

7.2 Simple Linear Regression

7.3 Nonlinear Equations Linear in the
Parameters

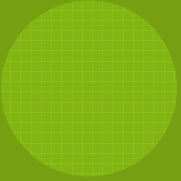
7.4 Equations with Nonlinear Parameters

7.5 Calibration to Dynamic Data

7.6 Evolutionary Techniques

7.7 Parameter Estimation Cautions





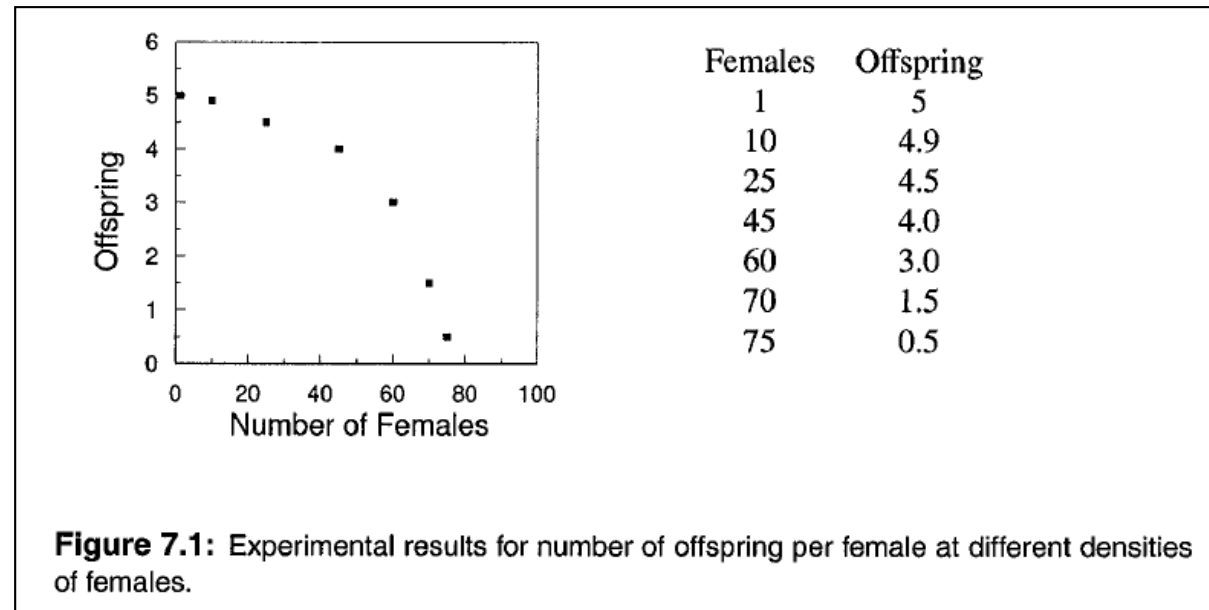
7.1 THE PROBLEM

- ◎ Every model that is used to make quantitative predictions contains parameters whose values must be specified.
- ◎ It is to be hoped that all of the parameters can be estimated in principle (i.e., have operational definitions), but even if this is true, performing the necessary experiments to estimate these values is often difficult in practice.



7.1 THE PROBLEM

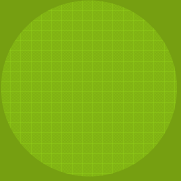
◎ A Model of Population Dynamics



$$y = k_1 + k_2 x^{k_3}$$

- ◎ We wish to know the estimates of the parameters of our function that provide the best fit to the data.





7.1 THE PROBLEM

- ◎ Most concepts of *best* involve minimizing the distance between the data and the function, summed over all data points.
- ◎ Some plausible candidate *measures of distance* are:
 1. Euclidean distance between function and data points.
 2. Square of vertical distance of data from the function. (the least squares criterion)
 3. The chi-square: square of vertical distance divided by the variance of the data. (the basis for the maximum likelihood estimates)
 4. Absolute value of vertical distance.
 5. Maximum distance of any one data point from the function as measured by one of the previous methods.



7.1 THE PROBLEM

- ◎ We wish to find the parameter values that provide the "*best*" fit to a particular data set. The statistical model we use is:

$$y_{ij} = f(x_i, p_k) + \varepsilon_i$$

- ◎ The standard definition of best is the least-squared difference, which attempts to minimize the error term:

$$\min \sum_i \varepsilon^2 = \min \left(\sum_i (y_{ij} - f(x_i, p_k))^2 \right)$$

- ◎ If we wish only to obtain a good fit with a function that passes through as many points as possible, then a cubic spline fit would be a good choice.



7.2 SIMPLE LINEAR REGRESSION

- ◎ One of the simplest functions we can attempt to fit to data is the linear function ($y = mx + b$), where m is the slope and b is the intercept.
- ◎ Static Applications: The easiest case to which linear regression applies is a simple experiment with a single independent variable. This is a classical application of linear regression in which the slope and intercept are the parameters of interest.



7.2 SIMPLE LINEAR REGRESSION

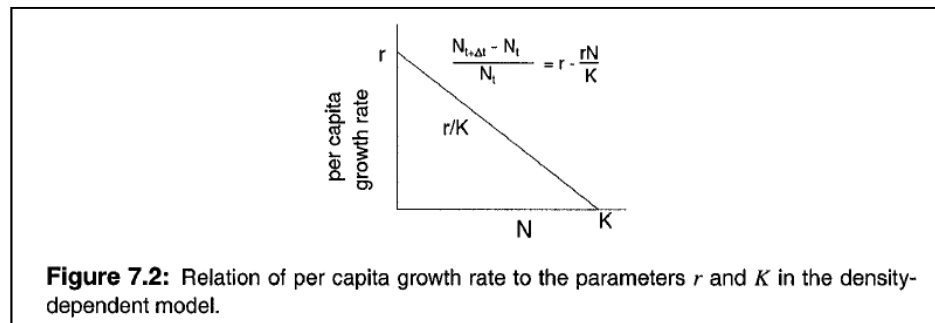
Dynamic Applications

- The density-independent model is itself a linear equation with the slope equal to r .

$$\frac{dN}{dt} = rN$$

- The density-dependent model: by dividing both sides by N we produce a new dependent variable $1/N \cdot dN/dt$ that is a linear function of N and has a negative slope.

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K} \right)$$



7.2 SIMPLE LINEAR REGRESSION

- ◎ Linear Regression on Transformed Equations
 - ◎ Regardless of the source of the data for regression (i.e., from static experiments or dynamic data), often the relations are nonlinear. In these cases, we may be able to transform the equation to a linear form.
 - ◎ ***Division by a variable***: This method was shown above when we created the per capita growth rate by dividing both sides of the differential equation by N . The idea is to reduce a squared term to a linear one.



7.2 SIMPLE LINEAR REGRESSION

- ◎ Linear Regression on Transformed Equations
 - ◎ **Logarithms:** Power functions are expressions in which the parameter to be estimated is part of the power of a constant or independent variable. These equations can be made linear by a log transform. For example,

$$y = Ax^b$$
$$\log y = \log A + b \log(x) \quad (7.1)$$

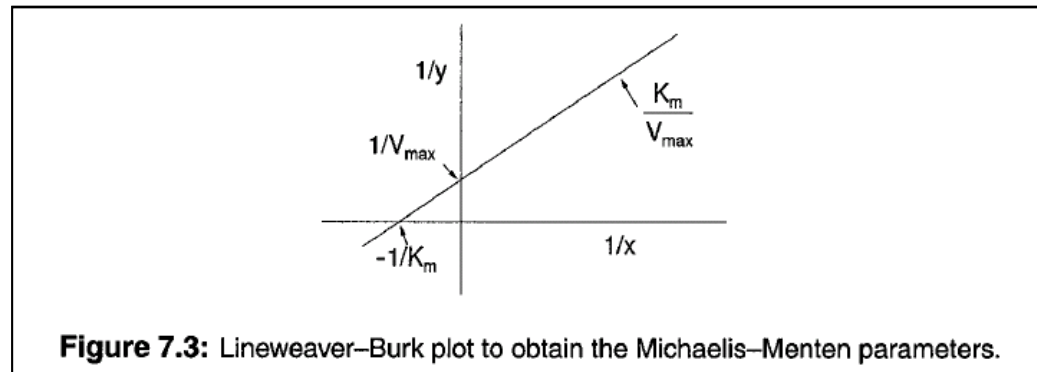


7.2 SIMPLE LINEAR REGRESSION

- ◎ Linear Regression on Transformed Equations
 - ◎ **Inverses:** Hyperbolic functions can be linearized by inverting the function. A famous example is the Michaelis-Menten relation for enzyme kinetics:

$$y = \frac{Ax}{B + x} \quad (7.2)$$

$$\frac{1}{y} = \frac{B}{A} \frac{1}{x} + \frac{1}{A} \quad (7.3)$$



7.2 SIMPLE LINEAR REGRESSION

⊙ Problems with Transformations

- ⊙ The important linear regression assumptions to satisfy are error *normality* and *homoscedasticity*. Merely straightening a curved line does not ensure that these assumptions are satisfied.
- ⊙ More advanced and better methods are commonly available in easy-to-use desktop computer statistical packages.
- ⊙ Linear regression can estimate only two parameters. For example, the sigmoid curve and its linear transformation is

$$y = \frac{A}{1 + Be^{Cx}} \quad (7.4)$$

$$\ln\left(\frac{A}{y} - 1\right) = \ln(B) + Cx$$

one of these must be assumed in order to estimate the other two.



7.2 SIMPLE LINEAR REGRESSION

◎ Problems with Transformations

- ◎ Inversion transformations can produce clustering of the resulting transformed data; this can produce spurious statistical correlations between the variables.
- ◎ Inverse transformations turn small numbers into large numbers. Often, the measurement of small quantities has large relative errors. These errors will be magnified after transformation.
- ◎ Since the log of a number less than or equal to 0.0 is undefined, logarithms can require that data be discarded or transformed prior to taking the logarithm.
- ◎ To make the parameter values stated in their original (untransformed) units requires that we "detransform" the numbers (e.g., take the anti-log of the intercept). Sometimes this *detransformation* will produce biased results.



7.3 NONLINEAR EQUATIONS LINEAR IN THE PARAMETERS

- There are powerful analytical techniques for estimating parameters in a special class of nonlinear functions characterized by being linear in the parameters. This means that although the equation is nonlinear with respect to the independent variable, the parameter is not involved in a nonlinear expression.
- Some examples of equations that are nonlinear in the parameters are:
$$y = \frac{ax}{b+x} \quad y = a \exp(bx) \quad y = ax^b$$
- If the equations are linear in the parameters, we can use several analytical techniques (*nonlinear* or *polynomial* regression). If they are nonlinear, we must use iterative techniques.



7.3 NONLINEAR EQUATIONS LINEAR IN THE PARAMETERS

◎ Multiple Linear Regression

- ◎ If the equation can be represented as a sum of terms, each of which is linear in the parameters (such as a polynomial equation), then **multiple linear regression** can be used to estimate the parameters. For example, if the equation is:

$$y = a + bx + cx^3$$

we notice that if we consider x^3 to be a separate variable (call it w , for example), then the equation is linear, and any of several software packages that can perform multiple linear regression will estimate c .



7.3 NONLINEAR EQUATIONS LINEAR IN THE PARAMETERS

◎ Polynomial Regression

- ◎ The general model for the relation of an observed dependent variable to a function evaluated at various observed independent variable points is

$$y_{ij} = f(x_i, p_k) - \varepsilon_i \quad (7.5)$$

- ◎ To implement the least-squares criterion, we wish to choose the p_k in order to minimize the sum of squared errors (ε_i in Eq. 7.5) over all the x_i observations. That is, we want the p_k such that

$$\min \sum_i \varepsilon^2 = \min \sum_i (f(x_i, p_k) - y_{ij})^2 \quad (7.6)$$



7.3 NONLINEAR EQUATIONS LINEAR IN THE PARAMETERS

◎ Polynomial Regression - Derivation

Take the following function as an example:

$$f(x_i, p_k) = A + Bx_i + Cx_i^2$$

where the problem is to find A, B, and C that satisfy our minimization criterion.

$$\varepsilon_i = (A + Bx_i + Cx_i^2) - y_i$$

$$g = \sum_i \varepsilon_i^2 = \sum_i ((A + Bx_i + Cx_i^2) - y_i)^2$$

after expanding,

$$g = \sum_i [(A^2 + 2ABx_i + 2ACx_i^2 - 2Ay_i) + (B^2x_i^2 + 2BCx_i^3 - 2Bx_iy_i) + (C^2x_i^4 - 2Cx_i^2y_i + y_i^2)]$$



7.3 NONLINEAR EQUATIONS LINEAR IN THE PARAMETERS

◎ Polynomial Regression - Derivation

Recall from calculus that the minima and maxima of functions relative to a variable can be found by setting the derivative of the function to 0. We wish to minimize g with respect to three "variables" (A , B , and C) simultaneously.

$$\begin{aligned}\frac{\partial g}{\partial A} &= \sum_i (2A + 2Bx_i + 2Cx_i^2 - 2y_i) \\ &= 2A \sum_i 1 + 2B \sum_i x_i + 2C \sum_i x_i^2 - 2 \sum_i y_i\end{aligned}$$

$$\frac{\partial g}{\partial B} = 2A \sum_i x_i + 2B \sum_i x_i^2 + 2C \sum_i x_i^3 - 2 \sum_i x_i y_i$$

$$\frac{\partial g}{\partial C} = 2A \sum_i x_i^2 + 2B \sum_i x_i^3 + 2C \sum_i x_i^4 - 2 \sum_i x_i^2 y_i$$



7.3 NONLINEAR EQUATIONS LINEAR IN THE PARAMETERS

◎ Polynomial Regression - Derivation

The error function g will be minimized at those A , B , C that cause each of the above partial derivatives to be 0. Therefore, we set the partials to zero to get three equations in three unknowns:

$$An + B \sum_i x_i + C \sum_i x_i^2 = \sum_i y_i \quad (\partial g / \partial A = 0)$$

$$A \sum_i x_i + B \sum_i x_i^2 + C \sum_i x_i^3 = \sum_i x_i y_i \quad (\partial g / \partial B = 0)$$

$$A \sum_i x_i^2 + B \sum_i x_i^3 + C \sum_i x_i^4 = \sum_i x_i^2 y_i \quad (\partial g / \partial C = 0)$$



7.3 NONLINEAR EQUATIONS LINEAR IN THE PARAMETERS

Polynomial Regression - Derivation

Equations such as these can be easily solved once they are re-written in matrix notation:

$$\begin{pmatrix} \sum_i y_i \\ \sum_i x_i y_i \\ \sum_i x_i^2 y_i \end{pmatrix} = \begin{pmatrix} n & \sum_i x_i & \sum_i x_i^2 \\ \sum_i x_i & \sum_i x_i^2 & \sum_i x_i^3 \\ \sum_i x_i^2 & \sum_i x_i^3 & \sum_i x_i^4 \end{pmatrix} \begin{pmatrix} A \\ B \\ C \end{pmatrix}$$

D = **S** **P**

Using matrix operations, we do this by pre-multiplying both sides of the equation by the inverse of **S** (denoted **S**⁻¹)

$$\mathbf{S}^{-1}\mathbf{D} = \mathbf{S}^{-1}\mathbf{S}\mathbf{P} = \mathbf{I}\mathbf{P} = \mathbf{P}$$



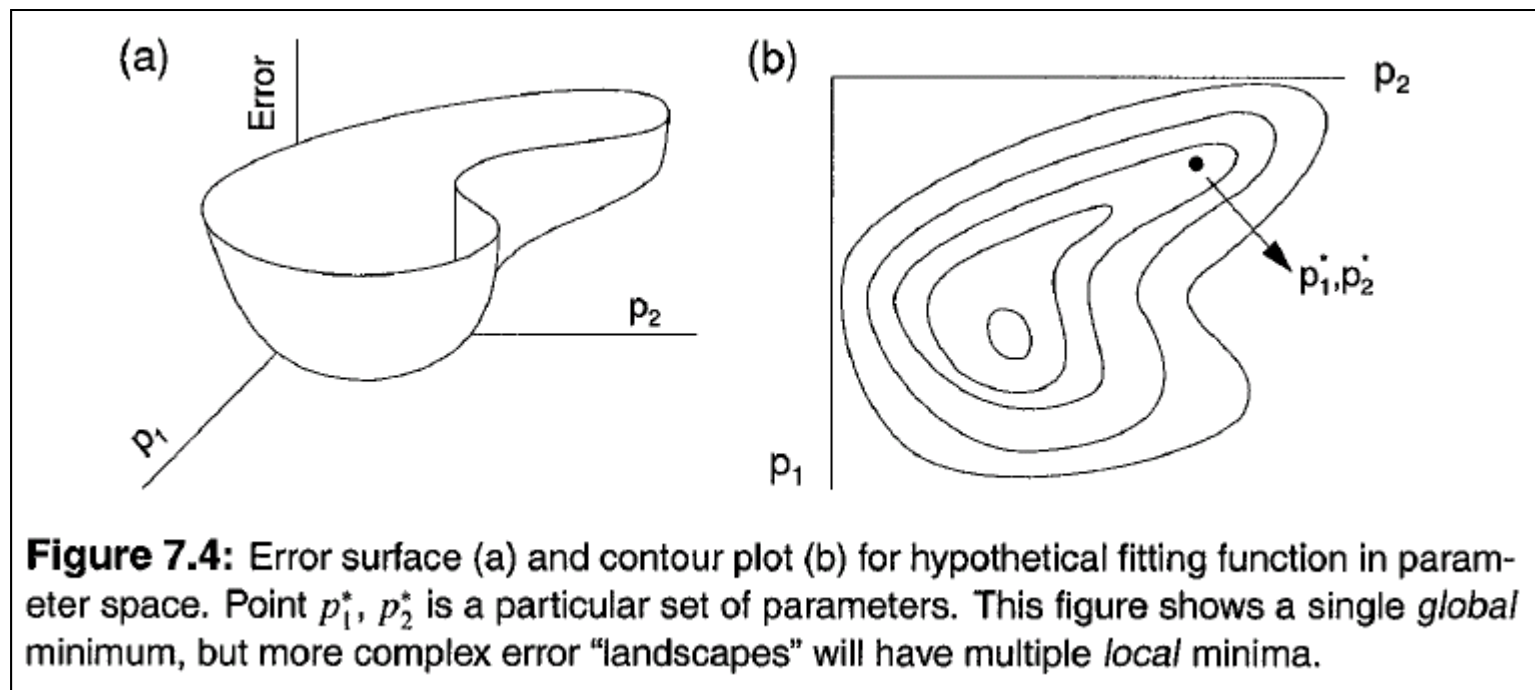
7.4 EQUATIONS WITH NONLINEAR PARAMETERS

- ◎ Some equations are not linear in the parameters and cannot or should not be transformed. Iterative methods must be used to estimate their parameters.
- ◎ There are two major approaches for iterative methods: *curvature-based* and *derivative-free*.
- ◎ The general problem in parameter estimation is to find the minimum point (i.e., the combination of parameters that corresponds to minimum error).
- ◎ Iterative methods start at some arbitrary point in the space and move from a parameter combination corresponding to large error to a combination with small error.



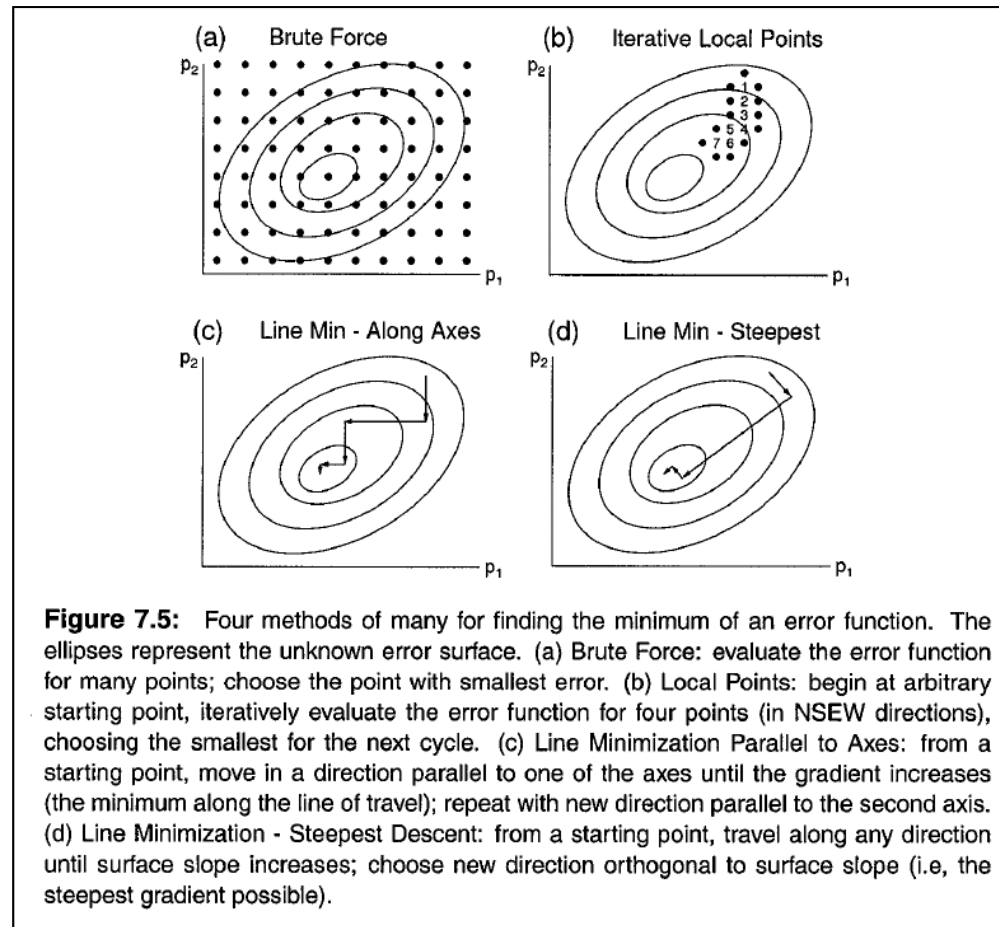
7.4 EQUATIONS WITH NONLINEAR PARAMETERS

- ⊙ The iterative algorithms move down the slope of the surface stopping only when the current parameter set is sufficiently close to the minimum.



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ Methods for Finding the Minimum of an Error Function



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ Gradient Methods

- ◎ For gradient methods, the direction of travel is based on the gradient of the slope, which is orthogonal to the previous direction that brought the current iteration to the line minimum.
- ◎ Gradient iterative methods, as a class, calculate the slope and curvature of the surface at the current set of parameters using a *Taylor approximation* and base the direction to change parameters on the direction of greatest change in the error surface.
- ◎ The gradient methods usually require either that the modeler provide the derivative of the function to be fit, or that the derivatives be numerically approximated.



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ Four Important Gradient Methods

- ◎ **Gauss** This method truncates *the Taylor series at the first-order terms*. In other words, it approximates the surface at the current solution point to a flat surface. This method requires that a matrix composed of first-order derivatives be computed and inverted.
- ◎ **Newton-Raphson** This method is similar to Gauss' method, but approximates the surface to a quadratic function by truncating the *Taylor series after the second-order terms*. This requires that a complex matrix of first- and second-order derivatives be computed and inverted. It has an explicit step-size parameter.
- ◎ **Steepest Descent** This is a simplification of the Newton-Raphson method. It eliminates the second-order derivatives and the matrix inversion, but retains the step-size parameter.
- ◎ **Levenberg-Marquardt (LM)** This method combines steepest descent with second-order derivatives. It is one of the most popular methods.



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

- ◎ An Example of Two-parameter Fitting
 - ◎ The basic idea is to iteratively change both parameters simultaneously:

$$\begin{pmatrix} p_{1,i+1} \\ p_{2,i+1} \end{pmatrix} = \begin{pmatrix} p_{1,i} \\ p_{2,i} \end{pmatrix} + \begin{pmatrix} \Delta p_{1,i} \\ \Delta p_{2,i} \end{pmatrix} \quad (7.7)$$
$$\mathbf{p}_{i+1} = \mathbf{p}_i + \Delta \mathbf{p}_i$$

- ◎ The problem is to compute a good value for $\Delta \mathbf{p}_i$. The LM method tries to use both the gradient (slope) of the error surface as well as its curvature to estimate $\Delta \mathbf{p}_i$.



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

- ◎ An Example of Two-parameter Fitting
 - ◎ In this two-dimensional case, the gradient is a vector with two elements, one for each parameter. The curvature is the slope of the slopes in all the directions. This Hessian, or curvature, matrix is

$$\mathbf{C} = \begin{pmatrix} \frac{\partial^2 \varepsilon}{\partial p_1 \partial p_1} & \frac{\partial^2 \varepsilon}{\partial p_1 \partial p_2} \\ \frac{\partial^2 \varepsilon}{\partial p_2 \partial p_1} & \frac{\partial^2 \varepsilon}{\partial p_2 \partial p_2} \end{pmatrix} \quad (7.8)$$



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ An Example of Two-parameter Fitting

- ◎ For steepest descent method:

$$\mathbf{p}_{i+1} = \mathbf{p}_i - \lambda_i \nabla \varepsilon \quad (7.9)$$

where λ is a constant that determines the size of the step to take.

- ◎ By using the relation that the product of the second derivative of errors and a finite unit of parameter distance will be approximately the first derivative of errors, the LM method can be formulated as:

$$\mathbf{C} \Delta \mathbf{p} = \nabla \varepsilon$$

$$\Delta \mathbf{p} = \mathbf{C}^{-1} \nabla \varepsilon$$

$$\mathbf{p}_{i+1} = \mathbf{p}_i - \mathbf{C}^{-1} \nabla \varepsilon \quad (7.10)$$



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ An Example of One-Dimension Search

Suppose the error surface was exactly a quadratic function:

$$\varepsilon = 0.1p_1^2 - 2p_1 + 10$$

The derivatives needed are:

$$\frac{d\varepsilon}{dp_1} = -2 + 0.2p_1 \quad (7.11)$$

$$\frac{d^2\varepsilon}{d^2p_1} = \varepsilon'' = 0.2 \quad (7.12)$$

If we use only steepest descent, arbitrarily choose $\lambda = 1$, and start with initial guess $p_{1,0} = 20$, Eq. 7.9 gives the parameter value in next iteration as:

$$p_{1,1} = 20 - (1)[-2.0 + (0.2)20] = 18$$

whereas using curvature as defined in Eq. 7.10 gives:

$$p_{1,1} = 20 - (1 / 0.2)[-2.0 + (0.2)20] = 10$$

Using Eq. 7.11, we see that the minimum is exactly at $p_1 = 10$.



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ The Levenberg-Marquardt (LM) Method

```
function LM_Fitting
%% create the first half of the data
xdata = 0:.01:1;
ydata = 3.0 ./ (1.0 + 2.5 * exp(4.65 * xdata)) + randn(size(xdata)) * .05;

%% call |LSQNONLIN|
init_guess = [3.5 2.3 4.9]
parameter_hat = lsqnonlin(@mycurve, init_guess, [], [], [], xdata, ydata)

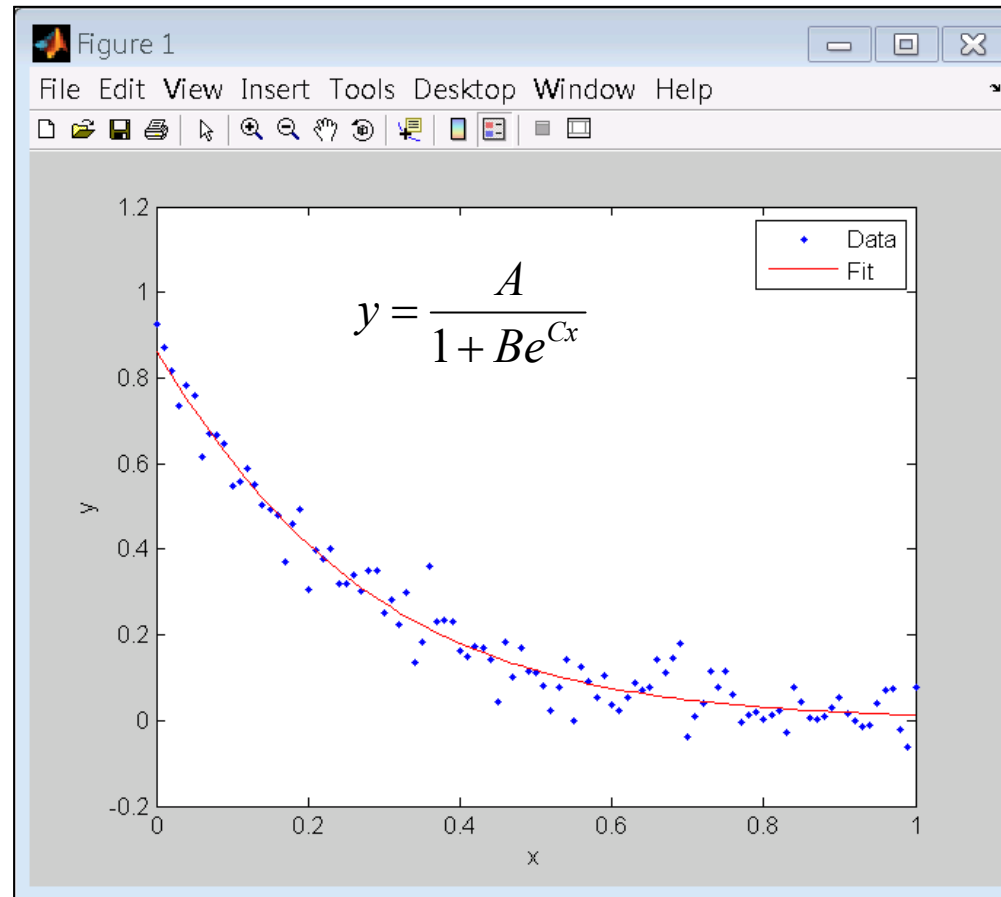
%% plot the original data and fitted function
plot(xdata, ydata, 'b.')
hold on
fitted = parameter_hat(1) ./ (1.0 + parameter_hat(2) * exp( parameter_hat(3) * xdata ));
plot(xdata, fitted, 'r')
xlabel('x'); ylabel('y')
legend('Data', 'Fit')

%% function that reports the error
function err = mycurve(parameter, real_x, real_y)
fit = parameter(1) ./ (1.0 + parameter(2) * exp( parameter(3) * real_x ));
err = fit - real_y;
end
end
```



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ The Levenberg-Marquardt (LM) Method



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ The Levenberg-Marquardt (LM) Method

Python Code

```
import numpy as np
from scipy import optimize
import matplotlib.pyplot as plt

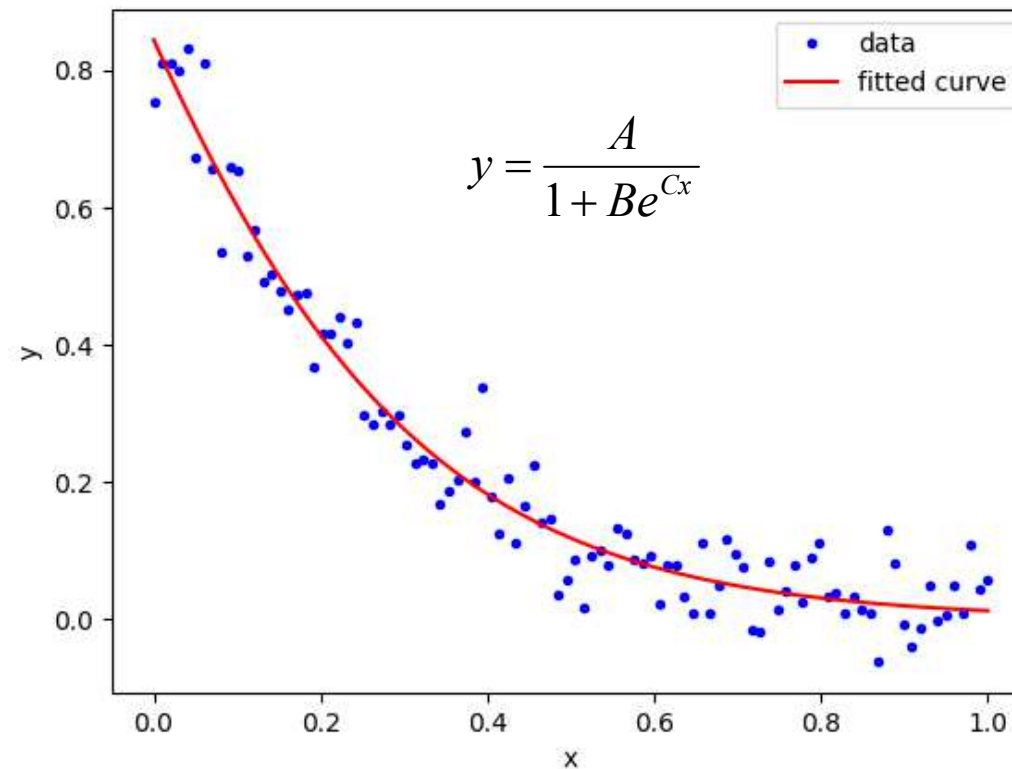
# let's generate x and y, and add some noise into y
xdata = np.linspace(0.0, 1.0, 100, endpoint=True)
ydata = 3.0/(1.0+2.5*np.exp(4.65*xdata)) + np.random.normal(loc=0.0, scale=0.05, size=xdata.size)
# initial guess of the parameters (exact parameter is [3.0, 2.5, 4.65])
parameter = [2.9, 2.3, 4.9]
# define the model function
def model_func(xdata, a, b, c):
    y = a / (1.0 + b*np.exp(c*xdata))
    return y
# Nonlinear regression using the LM method with initial guess p0
parameter_hat, cov = optimize.curve_fit(model_func, xdata, ydata, method='lm', p0=parameter)
#parameter_hat, cov = optimize.curve_fit(model_func, xdata, ydata, method='trf', bounds=([2.5, 2.0, 4.0], [3.5, 3.0, 5.0]),
p0=parameter) # This line is for testing with another algorithm. See scipy.optimize.curve_fit.
print('The fitted model function parameters are: ', parameter_hat[0], parameter_hat[1], parameter_hat[2])

# Calculate the fitted curve using the found parameters
predict = model_func(xdata, parameter_hat[0], parameter_hat[1], parameter_hat[2])
# plot the original data and fitted function
plt.plot(xdata, ydata, 'b.')
plt.plot(xdata, predict, 'r')
plt.xlabel('x')
plt.ylabel('y')
plt.legend(['data', 'fitted curve'], loc=1)
plt.show()
```



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ The Levenberg-Marquardt (LM) Method



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ Direct Methods

- ◎ Because of the computational cost of numerically approximating derivatives and performing matrix inversion, *direct methods* are an attractive alternative.
- ◎ *Direct methods* do not require derivatives and choose the direction for the next move by directly evaluating the error surface in the neighborhood of the current point (Fig. 7.5b).
- ◎ The main disadvantages of the method of Fig. 7.5b are that it examines values in a fixed neighborhood and it will zigzag.



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ Nelder-Mead Simplex Method

- ◎ This parameter estimation method is based on moving a geometric object (the simplex) through parameter space until the object encloses the best estimate.
- ◎ A simplex is a polygonal figure with one vertex more than the dimensions of the space in which it is embedded.
- ◎ The simplex method is an algorithm that alters the location of the simplex in parameter space so that when the stopping criterion is satisfied, the "best" values of the parameters are contained within the edges of the simplex.



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ Nelder-Mead Simplex Method

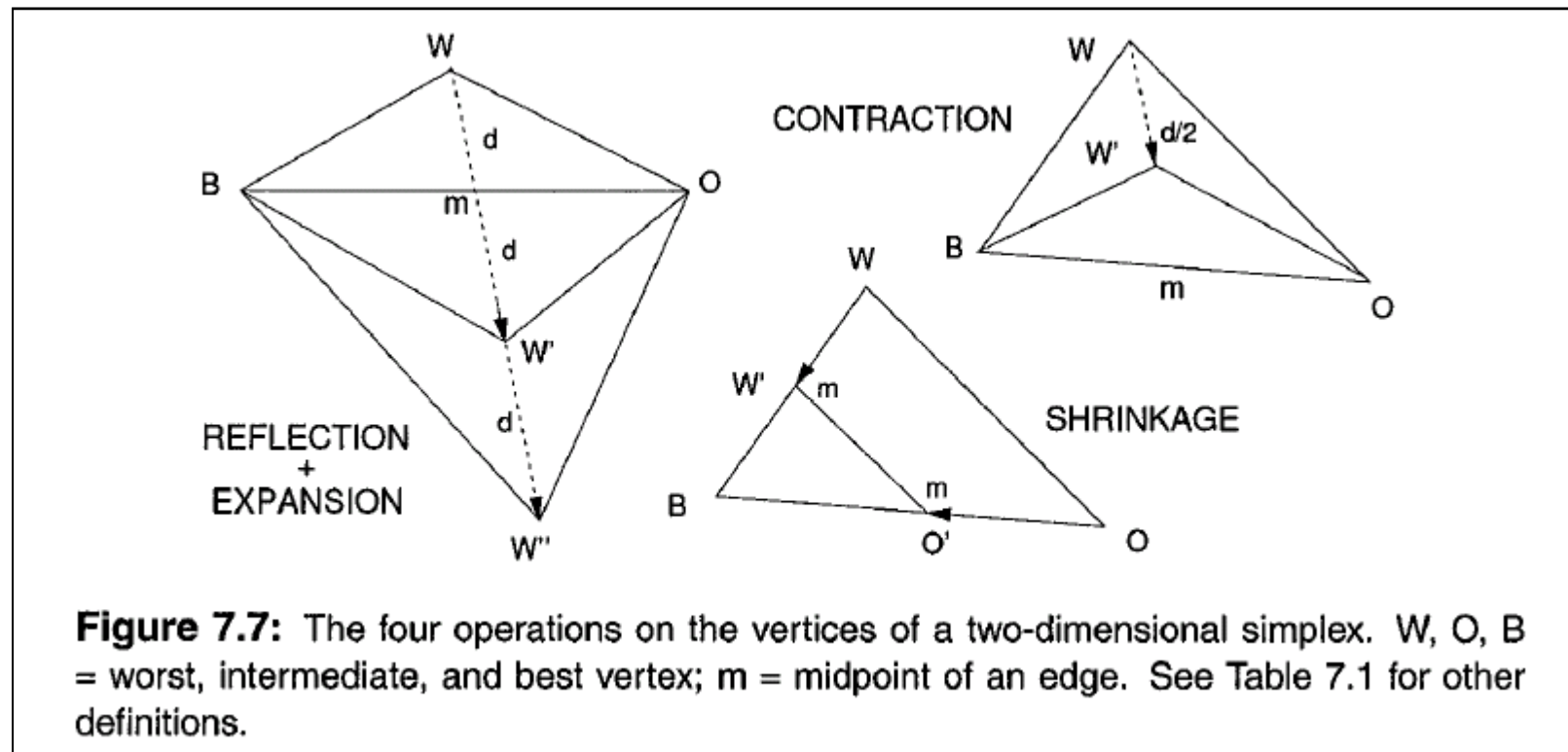
Table 7.1: Fundamental operations on a simplex (see Fig. 7.7).

Reflection	Extend a line d units long from W to the midpoint of the B – O edge and d units beyond. The end of the line $2d$ units long is the trial vertex (W').
Expansion	If W' is an improvement, continue the extension of the line another d units in the same direction to W'' .
Contraction	If reflection shows no improvement, extend a line $d/2$ units long from W to the midpoint of the B – O edge. Create a new vertex (W') at this point.
Shrinkage	If none of the above, create two new vertices, one at the midpoint of the B – O edge and the other at the midpoint of the B – W edge.



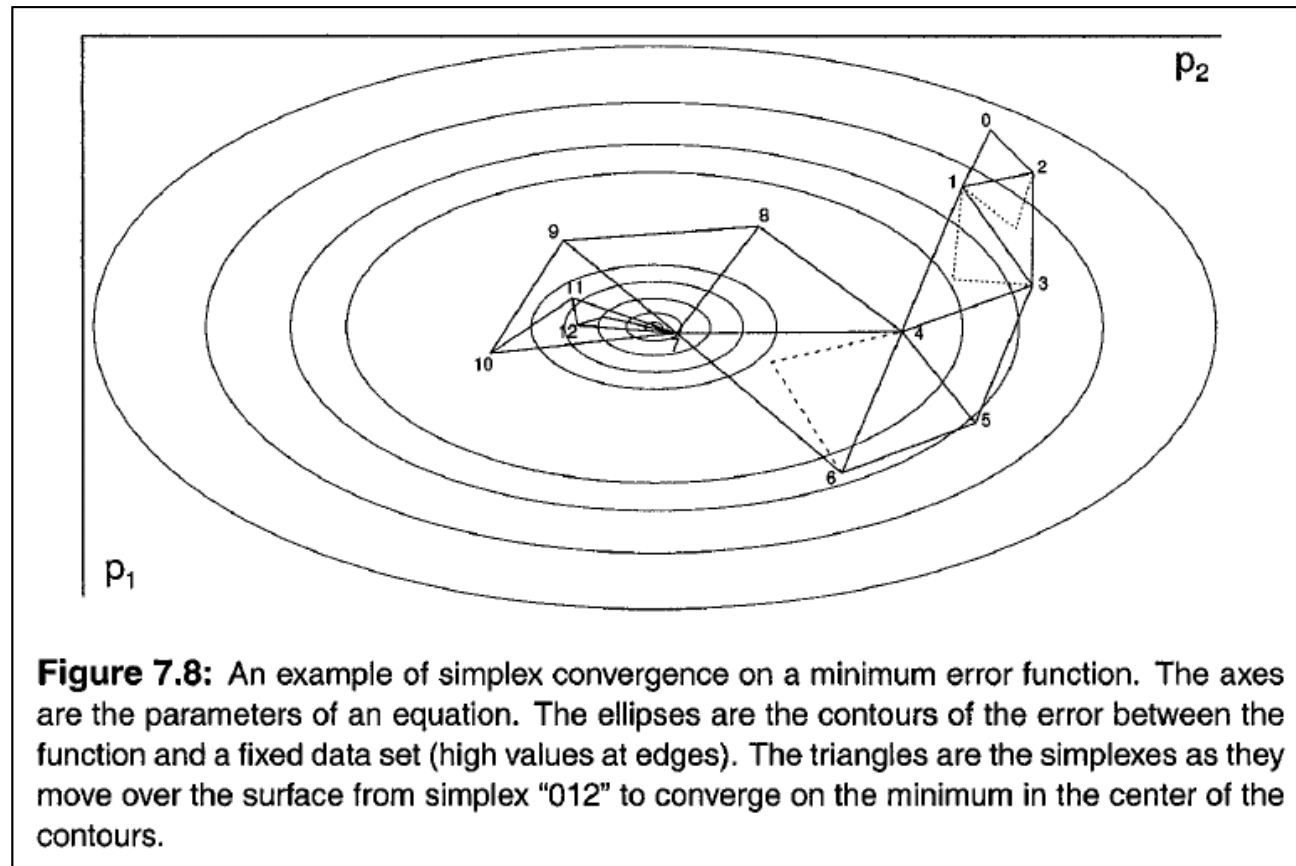
7.4 EQUATIONS WITH NONLINEAR PARAMETERS

☉ Nelder-Mead Simplex Method



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ Nelder-Mead Simplex Method



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ Nelder-Mead Simplex Method - Example

```
%% Optimal Fit of a Non-linear Function
% First, create some sample data and plot it.

t = (0:.1:2)';
y = [5.8955 3.5639 2.5173 1.9790 1.8990 1.3938 1.1359 1.0096 1.0343 ...
     0.8435 0.6856 0.6100 0.5392 0.3946 0.3903 0.5474 0.3459 0.1370 ...
     0.2211 0.1704 0.2636]';
plot(t,y,'ro'); hold on; h = plot(t,y,'b'); hold off;
title('Input data'); ylim([0 6])

%   y = C(1)*exp(-lambda(1)*t) + C(2)*exp(-lambda(2)*t)

type fitfun
start = [1;0];
% We use an anonymous function to pass additional parameters t, y, h to the
% output function.
outputFcn = @(x,optimvalues,state) fitoutputfun(x,optimvalues,state,t,y,h);
options = optimset('OutputFcn',outputFcn,'TolX',0.1);
estimated_lambda = fminsearch(@(x)fitfun(x,t,y),start,options)
displayEndOfDemoMessage(mfilename)
```



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

◎ Nelder-Mead Simplex Method - Example

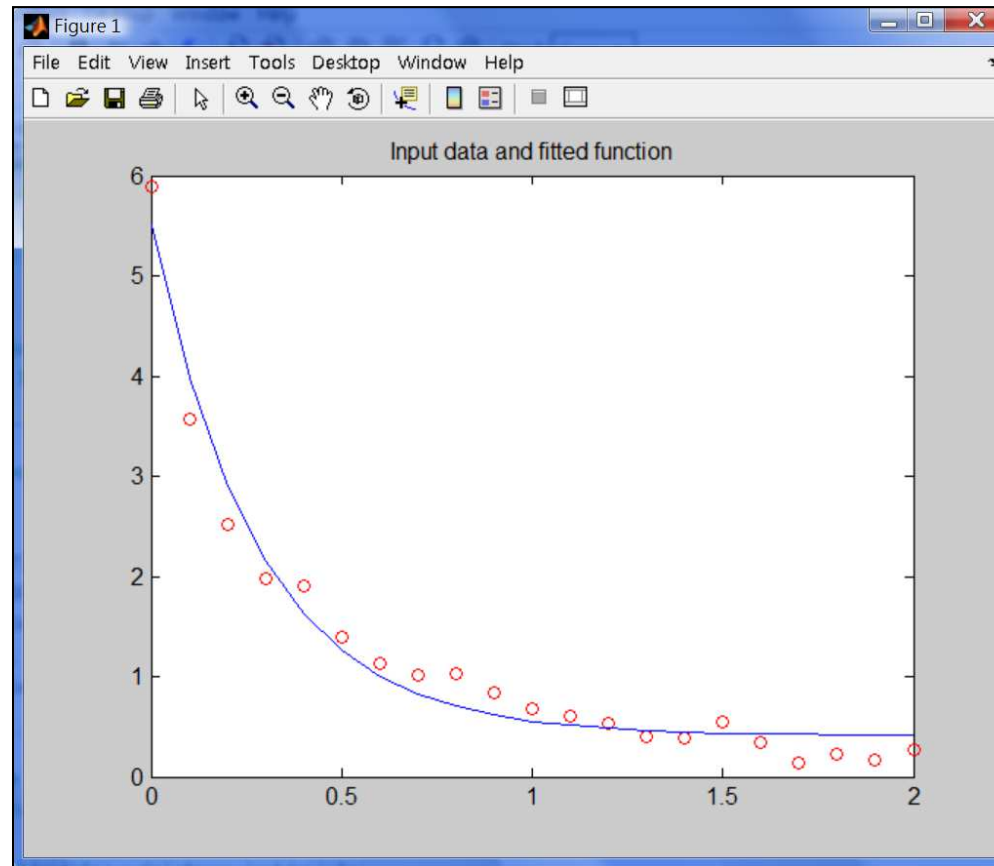
```
function err = fitfun(lambda,t,y)
%FITFUN Used by FITDEMO.
% FITFUN(lambda,t,y) returns the error between the data and the values
% computed by the current function of lambda.
%
% FITFUN assumes a function of the form
%      y = c(1)*exp(-lambda(1)*t) + ... + c(n)*exp(-lambda(n)*t)
% with n linear parameters and n nonlinear parameters.

A = zeros(length(t),length(lambda));
for j = 1:length(lambda)
    A(:,j) = exp(-lambda(j)*t);
end
c = A\y;
z = A*c;
err = norm(z-y);
```



7.4 EQUATIONS WITH NONLINEAR PARAMETERS

🎯 Nelder-Mead Simplex Method - Example



7.5 CALIBRATION TO DYNAMIC DATA

- ◎ Another approach to fitting parameters in a dynamic model is to find a set of parameters that minimize the sum of errors between the dynamic model output (e.g., numbers vs time) and similar observed dynamic trajectories over the entire time period simulated.
- ◎ There are two cases to consider:
 1. The function to fit is an analytical solution to a differential equation.
 2. The function to fit is the results of a simulation model.



7.5 CALIBRATION TO DYNAMIC DATA

◎ An Example of Density-Dependent Growth Model

$$\begin{aligned}\frac{dN}{dt} &= rN \left(1 - \frac{N}{K}\right) \\ N(t) &= \frac{K}{1 + e^{\beta - rt}}\end{aligned}\tag{7.13}$$

where r is maximum per capita growth rate, K is carrying capacity, and β is related to the starting population size $[N(0)]$. We can estimate all three parameters by fitting the function $N(t)$ to experimental data consisting of population size over time. Obviously, $N(t)$ is nonlinear in the parameters so we must use one of the techniques for nonlinear regression (transformation, gradient or direct methods).



7.5 CALIBRATION TO DYNAMIC DATA

◎ An Example of Linear System Model

$$\frac{dx}{dt} = ax + by \qquad \frac{dy}{dt} = cx + dy$$

the dynamics $[x(t)$ and $y(t)]$ can be written as a sum of exponentials. In other words, we can find an *analytical solution* whose parameters can be estimated using the methods described above. In this special case of systems of linear differential equations, the parameter estimation problem is known as ***system identification***.

- ◎ When a differential model can not be solved analytically, the calculation of derivatives needed by some methods becomes complicated. Therefore, the direct method are effective on this problem.



7.6 EVOLUTIONARY TECHNIQUES

- ◎ Parameter estimation is an optimization problem, and radically new approaches have been introduced recently based on analogies with the evolution of biological systems.
- ◎ The new methods are members of a loose family of algorithms called *evolutionary computation*.
- ◎ The basic idea applied to parameter estimation is that the parameter space is searched by a large set of "organisms" that are defined by their position in the space. Their fitness is the value of the error function at that point in parameter space.
- ◎ Organisms with low fitness (large error) are discarded. Surviving organisms mate and produce slightly different offspring by combining the positions of the two parents to form a new location in parameter space. This process is repeated until organisms do not show further improvement.



7.7 PARAMETER ESTIMATION CAUTIONS

◎ All Methods

1. Beware of transformations. Nonlinear regression or iterative methods are preferred.
2. Examine your data for obvious outliers. You may need to filter the data or apply some other method for eliminating extreme data points.
3. Beware of extrapolating beyond your data. Some situations in some methods can also make interpolating between datum points dangerous. Rational functions should not be used for data sets with multiple observations.



7.7 PARAMETER ESTIMATION CAUTIONS

◎ All Methods

4. Beware of using a simple statistical index (e.g., r^2) to determine the function to use. An equation with sufficiently large numbers of parameters can be fit to match every little jog in a noisy data set with high r^2 , but may fail to reveal a simpler representation.
5. Use a graphics package to view your data and fitted curve. Be suspicious of any obvious departures. We wish to obtain a simple and general description of the observations. Simplicity in the form of equations with small numbers of parameters is usually preferable to complicated equations with a good fit to a particular dataset. The equation is the object of interest, not the r^2 .



7.7 PARAMETER ESTIMATION CAUTIONS

© Iterative Methods

1. Non-evolutionary, iterative methods find only local minima. Use several starting points to search for the global minimum. The newer methods using evolutionary computation appear to be better at finding the global minima (or maxima).
2. Methods requiring derivatives can be slow and sensitive to the "roughness" of the error surface. Steep gradients and sudden reversals can cause numerical approximation of derivatives to go astray. Methods such as simplex that do not use derivatives are less sensitive to this. Test the results with several step sizes.



7.7 PARAMETER ESTIMATION CAUTIONS

◎ Iterative Methods

3. Most iterative methods do not give exact r^2 values. Approximate values can be obtained by *bootstrapping* or *by fitting a polynomial to the error function* after a good fit is found.
4. Most iterative methods use two stopping criteria: one based on the relative change in the residuals and the other a ceiling on the number of iterations performed. After the algorithm has stopped, verify that sufficient iterations were allowed to ensure that the first criterion (not number of iterations) was used to stop the search.

